

Build Day Guide — Network, SSH, Gaming VM

Goal for today: network up → SSH working → Windows VM with RTX 5080 passthrough → Sunshine streaming to the mini PC over Moonlight. Tags show who does each part: **PERSON AT SERVER** **PENN + CLAUDE (remote)** **FRIEND / MINI PC**

Router port map (MikroTik RB5009 — ports numbered on the front)

Port	What plugs in	Network
ether1	Cable from the home router (a LAN port) — this is the internet feed	WAN
ether2	Mini PC (Moonlight, in the TV room)	30 gaming
ether3	Backup / storage box (later)	20 servers
ether4	Server's BMC / IPMI management port (small separate port; later)	10 mgmt
ether5–7	Penn's laptop / any trusted computer	40 trusted
ether8	Server's main 10G network port (temporary — moves to SFP+ when module arrives)	trunk
SFP+ cage	Future: server's 10G link via S+RJ10 / 10Gtek module (on order)	later

Bold rows are needed today: internet into **ether1**, server into **ether8**, Penn's laptop in **ether5**, mini PC in **ether2** (mini PC can wait until Phase 4).

Phase 1 — Network + SSH **PERSON AT SERVER** (~10 min)

- 1 Plug in the cables** — home router → **ether1**; server's 10G port → **ether8**. The server has **two identical 10G ports** — if step 3's first ping fails, move the cable to the other one.
Also plug the APC UPS's USB cable into the server (any USB port) — the safe-shutdown service set up in Phase 2 talks to the UPS over it.
- 2 Log in at the server** — monitor + keyboard on the server. Username `root`, password from the Proxmox install.
- 3 Type these four lines** (Enter after each; straight quotes, exact spacing). **Each command is ONE line** — if it wraps on this page, keep typing on the same line:

```
sed -i 's|address .*|address 10.10.10.10/24|; s|gateway .*|gateway 10.10.10.1|' /etc/network/interfaces

sed -i '/bridge-fd/a\ bridge-vlan-aware yes\n bridge-vids 2-4094' /etc/network/interfaces

ifreload -a

ping -c 2 10.10.10.1 && ping -c 2 1.1.1.1
```

You want replies from **both pings**:

- `10.10.10.1` replies = server sees the router. If not → check the ether8 cable / try the other 10G port.
- `1.1.1.1` replies = internet works. If not → check the ether1 cable.

- 4 Type this one line — this is the SSH setup.** It downloads a script from the project repo that installs Penn's SSH public key into `/root/.ssh/authorized_keys`, fixes the hostname file, and prints a hardware report:

```
curl -fsSL https://raw.githubusercontent.com/melonmelonz/infra-scripts/main/scripts/host-bootstrap.sh | bash
```

It should end with `BOOTSTRAP DONE`.

Why this is all SSH needs: Proxmox already runs an SSH server. Adding the key means Penn's laptop is recognized and logs in with no password. Nothing else to install or configure.

- 5 Photo everything from `=== IP ===` down, send to Penn.** The GPU lines (`[10de:....]` codes) are the critical part — Phase 2 needs them.

If `=== IOMMU ===` printed nothing: reboot, press Del for BIOS, set **SVM** and **IOMMU** to Enabled (Advanced → CPU / AMD CBS), boot, redo step 4. While in BIOS also confirm **Above 4G Decoding** and **Resizable BAR** are Enabled (needed for GPU passthrough).

Phase 2 — Connect by SSH + host prep **PENN + CLAUDE** (~20 min + 1 reboot)

Connecting by SSH (from Penn's laptop, wired into ether5)

```
ssh root@10.10.10.10
```

- **No password prompt** — the key from Phase 1 step 4 handles it. First connect asks to trust the host fingerprint: type `yes`.
- **Laptop must be wired to ether5/6/7**. Home wifi cannot reach 10.10.10.x — the firewall blocks it by design. (This restriction disappears once Tailscale is up, below.)
- The Proxmox web interface also works now: `https://10.10.10.10:8006` (accept the certificate warning, login root).
- **If SSH asks for a password**: step 4 didn't complete — re-run the curl line at the console.
- **If SSH times out**: laptop not on ether5/6/7, or step 3's pings weren't actually green.

Host prep (Claude runs this; listed so the work is visible)

- Fill in `ansible/inventory.ini` (host = 10.10.10.10) and `vfio_pci_ids` in `ansible/group_vars/all.yml` with the GPU IDs from the Phase 1 photo — both the 5080 and 3080, each with its audio function.
- Run `ansible-playbook site.yml`. What it configures:
 - **VFIO** — kernel cmdline (`amd_iommu=on iommu=pt`), binds both NVIDIA cards to vfio-pci at boot, blacklists `nouveau/nvidia` on the host. The host never touches the GPUs; VMs own them.
 - **ZFS** — caps ARC cache at 8GiB so VM RAM is never squeezed.
 - **NUT** — APC UPS monitoring over the USB cable; clean host shutdown on power loss.
 - **Tailscale** — installs; activates when Penn supplies an auth key (free account at tailscale.com → Settings → Keys). After that, SSH works from anywhere, no ether5 cable.
- Reboot the host, then run `scripts/verify-passthrough.sh` on it — checks IOMMU active, both GPUs on vfio-pci, IOMMU group isolation, ARC limit, UPS responding. **Every line must say PASS before Phase 3.**

Phase 3 — Windows VM + RTX 5080 + Sunshine **PENN + CLAUDE** (~1–2 hrs, mostly Windows installing)

- **ISOs**: Windows 11 + virtio-win drivers, downloaded straight to the host (it has internet now).
- **VM shape**: q35 machine, UEFI (OVMF) + TPM 2.0 (Win11 requires it), CPU type `host`, 32GB RAM to start (locked in place — GPU passthrough pins it), disk on the NVMe ZFS pool, **network on VLAN 30 (gaming)** — same segment as the mini PC, which is exactly what makes Moonlight auto-discovery work.
- **Install order matters**:
 1. Install Windows using the emulated display (Proxmox web console), loading virtio drivers during setup.
 2. Enable RDP inside Windows (backup way in once the console goes away).
 3. Attach the 5080 (GPU + audio function, PCIe, all functions) to the VM; reboot it.
 4. Install NVIDIA drivers inside Windows — the 5080 is now a real GPU in the VM.
 5. Install **Sunshine** (the streaming host).

Headless GPU catch: a passthrough GPU with no monitor renders nothing, and Sunshine needs an active display.

Today's fix: install Virtual Display Driver (software, free). **Better permanent fix:** a ~\$7 **HDMI dummy plug** in the 5080 — order with the 10G module.

Phase 4 — Moonlight on the mini PC FRIEND / MINI PC (~10 min)

- Plug the mini PC into **ether2**. It lands on the gaming network (10.10.30.x) automatically.
- Install **Moonlight** (moonlight-game-streaming.org) on it.
- Open Moonlight — the Windows VM appears by itself (same network segment, auto-discovery). Click it, enter the pairing PIN shown by Sunshine on the VM (Penn reads it over RDP), done.
- The stream is switched in hardware inside the router — it never crosses VLANs or touches the router CPU. Latency is sub-millisecond on the LAN.

End state for today: friend launches Moonlight on the mini PC and plays games running on the 5080 in the server. Penn has SSH + web UI to the host (and from anywhere, once Tailscale has a key).

Shopping list (nothing blocks today)

- 10G SFP+ RJ45 module (10Gtek / S+RJ10 style) — on order. Arrives → goes in SFP+ cage, server cable moves to it, Claude flips one router setting.
- HDMI dummy plug (~\$7) — replaces the virtual display driver, plug into the 5080.